

# CS & IT ENGINEERING

## 'C' Programming

### Functions

Lecture No.- 01



By- Satya sir

# Recap of Previous Lecture



- Pointers and arrays

- Pointer to an Element of array Ex: `int *P;`

- Pointer to whole array Ex: `int (*P)[5];`

- Array of Pointers Ex: `int *P[5];`

- Pointer to Pointer

- 3-D array



# Topics to be Covered



- Practice of Q's
- Functions
  - Declaration Vs Definition Vs Calling
  - Caller and Callee
  - Function Prototypes



## Topic : Practice Session – GATE 2020

```
#Q. int pp(int a, int b) {  
    int arr[20];  
    int i, tot = 1, ex, len;  
    ex = a;  
    len = tob(b, arr);  
    for (i=0; i < len; i++) {  
        if (arr[i] == 1)  
            tot = tot * ex;  
        ex = ex * ex; // execute always  
    }  
    return (tot);  
}
```

Handwritten annotations for the code above:

- 3 above 'a', 4 above 'b' in the function signature.
- 0 1 2 3 ... 19 above the array index range.
- arr points to the array.
- ex = 3 (with a=3) and len = 3 (with b=4) are annotated.
- return 81 is underlined.

Consider the following C functions.

```
int tob(int b, int* arr) {  
    int i;  
    for (i=0; b>0; i++) {  
        if (b%2) arr[i] = 1;  
        else arr[i] = 0;  
        b = b/2;  
    }  
    return (i);  
}
```

Handwritten annotations for the code above:

- 4 above 'b' in the function signature.
- odd 'b' value True points to the condition `b%2`.
- return 3 is underlined.

| i=0                | i=1        | i=2        |
|--------------------|------------|------------|
| 4 > 0 True         | 2 > 0 True | 1 > 0 True |
| if() False         | if() False | if() True  |
| arr[0] = 0         | arr[1] = 0 | arr[2] = 1 |
| b = 2              | b = 1      | b = 0      |
| i=3<br>0 > 0 False |            |            |

The value returned by pp(3, 4) is 81.

## [MCQ]



#Q. Consider the following C program:

[GATE-CS-shift-I-24: 1M]

```
#include <stdio.h>
int main()
```

```
{
```

```
    int a = 6
```

```
    int b = 0;
```

```
    while (a < 10)
```

```
    {
```

```
        a = a / 12 + 1;
```

```
        a += b;
```

```
    }
```

```
    printf("%d", a);
```

```
    return 0;
```

```
}
```

Which one of the following statements is CORRECT?

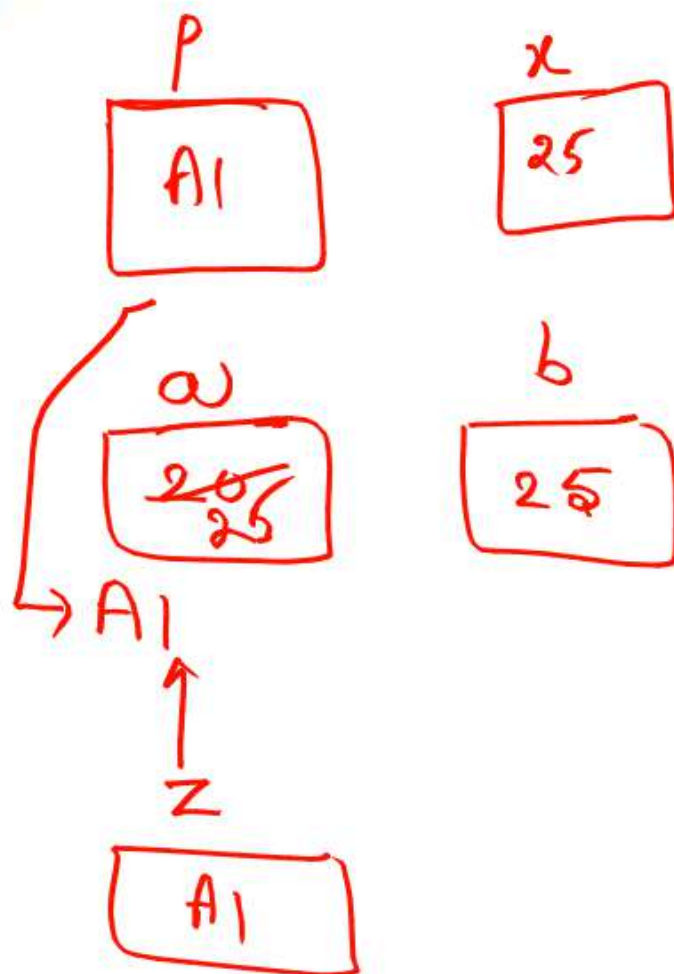
- ☒ A The program gets stuck in an infinite loop
- ☐ B The program prints 9 as output
- ☐ C The program prints 10 as output
- ☐ D The program prints 6 as output

$$\begin{array}{l} 6 < 10 \text{ True} \\ \left| \begin{array}{l} 1 < 10 \text{ True} \\ a = 0 + 1 = 1 \\ a = 1 \end{array} \right| \end{array}$$

a value will be  
always '1', 1 < 10 True  
always.



```
#include <stdio.h>
void foo(int *p, int x){
    *p=x;
}
int main(){
    int *z;
    int a = 20, b = 25;
    z = &a;
    foo(z,b);
    printf("%d",a);
    return 0;
}
```



The output of the given C program is 25. (Answer in integer)



GATE 2025 2M

Consider the following C program:

```
#include <stdio.h>
int gate (int n) {
    int d, t, newnum, turn;
    newnum = turn = 0; t=1;
    while (n>=t) t *= 10;
    t /=10;
    while (t>0) {
        d = n/t;
        n = n%t;
        t /= 10;
        if (turn) newnum = 10*newnum + d;
        turn = (turn + 1) % 2;
    }
    return newnum;
}
int main () {
    printf ("%d", gate(14362));
    return 0;
}
```

Handwritten calculations and variable states:

$n = 14362 \rightarrow 4362 \rightarrow 362 \rightarrow 62 \rightarrow 2$

| d                                                                  | t                                             | newnum                                    | turn                                  |
|--------------------------------------------------------------------|-----------------------------------------------|-------------------------------------------|---------------------------------------|
| <del>14362</del><br>2                                              | <del>100000</del><br>100000                   | <del>0446</del>                           | <del>0100</del>                       |
|                                                                    | $14362 \geq 1$<br>$t = 100000$                |                                           |                                       |
|                                                                    | $t = 100000/10 = 10000$                       |                                           |                                       |
| $d = 14362/10000 = 1$<br>$n = 14362 \% 10000 = 4362$<br>$t = 1000$ | $d = 4362/1000 = 4$<br>$n = 362$<br>$t = 100$ | $d = 362/100 = 3$<br>$n = 62$<br>$t = 10$ | $d = 62/10 = 6$<br>$n = 2$<br>$t = 1$ |
|                                                                    |                                               | $d = 2$<br>$n = 0$<br>$t = 0$             | 1 > 0 True                            |

turn sequence: turn=1, turn=0, turn=1, turn=0, turn=1

Final newnum: 46

46

The value printed by the given C program is 46. (Answer in integer)



### Function :

- A single statement (or) group of statements that performs a specific task (or) sub-task is said to be function.
- It is also known as Procedure sometimes.
- All statements inside function are kept under  $\{ \}$ , is known as Function Block (or) Body of function.
- 2 Types of functions :
  - 1) Pre defined (or) System-defined (or) Library functions
    - functions which are already defined, and stored in 'C' Library and organized as Header files.
    - Ex: `Printf()`, `scanf()`, `fclose()`, `strlen()`, `sizeof()`, `getch()` - - -
  - 2) User-defined functions Ex: `main()`



## Topic : Functions, Prototypes



User-defined function

Definition of function

Syntax:

return type    function name (arguments (or) Parameters\*)  
{  
    // Statement(s)  
}

Ex: 2

void foo(int x)

{

printf(".%.d", x\*x);

}

Ex: 3

void display()

{

printf("Welcome to Functions");

}

Ex: 1

```
int f(int a, int b)
{
    int c;
    c = a * b;
    return c;
}
```



## Topic : Functions, Prototypes



- 'C' compiler Processes only main() function. [Start Execution from main() and stop with main()].
- So, for any other functions to get executed/Processed, they need to be called (or) invoked either directly (or) Indirectly inside the main() function.

- Calling a function Syntax :

function name (arguments);

Ex:1 printf("%d", f(3,4));  
(OR)  
int x;  
x = f(3,4);

Ex: printf( ); scanf( );  
clrscr( ); getch( );

Ex:2 foo(9);

Ex: display( );

- Caller, Callee : A function, which call another function is Caller (or) Calling function  
A function, which is called by another function is Callee (or) Called function.



## Topic : Functions, Prototypes



### Flow of Execution

Ex: Line Numbers

```
1 int f1(int x, int y)
2 { int z;
3   z = x + y;
4   f2(z);
5   return z;
6 }
7 void f2(int x)
8 { printf("%d", x * x);
9 }
```

```
8 void main()
9 {
10   int i = 5, j = 7, k;
11   k = f1(5, 7);
12   printf("%d", k);
13 }
```

Order of Execution : 8, 9, 10 [1, 2, 3, 4 [6, 7], 5], 11



### Function Prototypes

- 1) Functions with argument(s), with return Value
- 2) Functions with argument, without return Value
- 3) Functions without argument, without return Value
- 4) Functions without argument, with return Value.

Ex:

```
//callee
Void display(int x)
{
    Printf("%d", x);
}
```

```
void main() //caller
{
    display(x);
}
```

With argument, without  
return value

```
//callee
Void display( )
{
    Printf("Hello");
}
```

```
Void main() //caller
{
    display( );
}
```

without arg, without  
return value

```
//callee
int fun(int x, int y)
{
    return x*y;
}
```

```
Void main() //caller
{
    Printf("%d", fun(5,7));
}
```

With argument, with  
return value

```
//callee
int fun( )
{
    int x, y;
    scanf("%d %d", &x, &y);
    return x*y;
}
```

```
Void main() //caller
{
    Printf("%d", fun());
}
```

Without argument,  
with return value



## 2 mins Summary

→ Functions, Types, Prototypes



### Declaration vs Definition vs Calling

→ when a function is called, before it is defined, Then Prototype declaration is needed.

→ `returntype function name(arguments type);`

Declaration: `int f(void);` (OR) `void f(int, int);` (OR) `int x(int);` (OR) `int* f(int*);`  
Examples

Ex: `void f(int);` → Declaration

`void f(int x)`  
`{`  
`printf("f.d", x);`  
`}` → Definition

`f(5);` → Calling



**THANK - YOU**